



SISTEMA ESPERTO IBRIDO PER LA DIAGNOSTICA FITOPATOLOGICA

Relazione di Progetto - Ingegneria della Conoscenza

Giuseppe Giannichi, 799184, g.giannichi@studenti.uniba.it

Link repository: https://github.com/Peppe0604/plant_diag.git

Anno Accademico: 2025-2026

1. Introduzione e Contesto del Dominio

L'agricoltura di precisione rappresenta una delle sfide più attuali nell'applicazione dell'Intelligenza Artificiale. Il dominio di interesse di questo progetto è la **fitopatologia diagnostica**, specificamente focalizzata sulle malattie degli alberi da frutto (Pesco, Mandarino, Albicocco).

problema affrontato nasce dalla difficoltà, per agricoltori non specializzati o hobbisti, di distinguere patologie con sintomatologie visive simili (es. deformazioni fogliari, macchie). Una diagnosi errata può portare all'uso improprio di fitofarmaci, con danni economici e ambientali.

L'obiettivo del progetto è sviluppare un **Sistema di Supporto alle Decisioni** ibrido che guidi l'utente nell'osservazione dei sintomi e fornisca una diagnosi accurata utilizzando due paradigmi di ragionamento complementari:

1. **Ragionamento Simbolico (Deduttivo):** Per diagnosi certe basate su protocolli fitosanitari standardizzati.
 2. **Ragionamento Probabilistico (Induttivo):** Per gestire l'incertezza e apprendere dai dati storici quando la sintomatologia è ambigua.
- **Dominio applicativo:** diagnosi per alcune colture (Pesco, Mandarino, Albicocco) e un sottoinsieme di malattie comuni (Bolla del pesco, Monilia, Cocciniglia, Gommosi varie, e 'Sano').
 - **Fatti / Sintomi:** si modellano 7 sintomi binari (presente/assente), es. foglie arricciate, macchie, presenza insetti, melata, gommosi, ecc.

Questi sono le feature usate dalla rete Bayesiana e interrogate nel sistema esperto.

2. Formato dataset

Ogni riga del CSV atteso ha la forma (intorni):

Specie,S1,S2,S3,S4,S5,S6,S7,Malattia dove:

- Specie è un codice intero (es. 0/1/2) che identifica la specie; è usato dal sistema esperto per impostare fatti contestuali;
- S1..S7 sono valori binari (0/1) dei 7 sintomi;
- Malattia è un intero che mappa alle classi (0 = Sano, 1 = Bolla, ...).

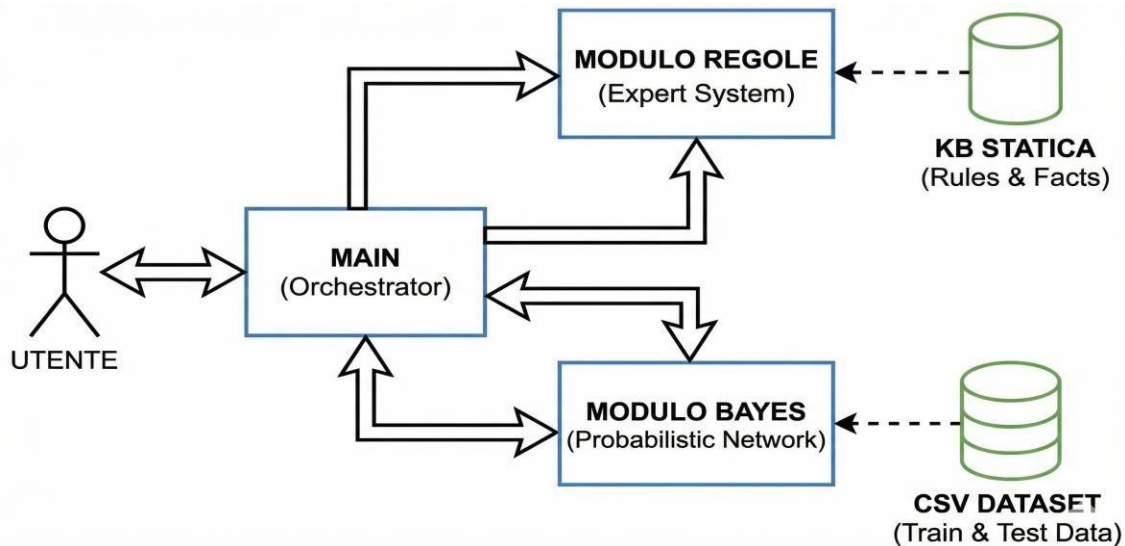
3. Ambiente di Sviluppo e Strumenti

Per garantire efficienza computazionale e portabilità, si è scelto di sviluppare il sistema interamente in **Linguaggio C (CGG Standard)**.

3.1 Motivazioni della scelta tecnologica

- **Performance e Controllo Memoria:** A differenza di linguaggi interpretati come Python, il C permette una gestione diretta della memoria e offre performance elevate.
- **Assenza di Librerie Esterne:** Il sistema è stato implementato "da zero", inclusi gli algoritmi di inferenza e calcolo matriciale.
- **Toolchain:**
 - **Editor:** *Vim* per la scrittura del codice sorgente.
 - **Compilatore:** *GCC (GNU Compiler Collection)* per la compilazione e il linking dei moduli.
 - **Formato Dati:** CSV (Comma Separated Values) per l'interscambio dei dataset di training e testing, garantendo interoperabilità.

4. Architettura del Sistema



Il software segue un'architettura modulare. Il `main.c` funge da orchestratore, permettendo all'utente di scegliere tra due sottosistemi indipendenti ma cooperanti.

- Architettura ibrida: combinare uno expert system (per conoscenza esplicita dell'esperto) e un classificatore statistico (Naive Bayes) per coprire sia regole certe sia pattern sottostanti nei dati.
- **Rete Bayesiana:**
 - Implementazione: modello Naive Bayes binario (assume indipendenza condizionale dei sintomi dato la malattia).
 - Rappresentazione: log-probabilità per prior (`log_priors`) e likelihoods (`log_likelihoods`) per stabilità numerica (evita underflow).
 - Smoothing: Laplace/Dirichlet con $\text{ALPHA} = 1.0$ per evitare probabilità nulle per eventi non osservati nel training (vedi `bayes_network.h`).
 - Inferenza: MAP (massima a posteriori) calcolata sommando i log-prior + somme dei log-likelihood (presa in considerazione anche dell'assenza dei sintomi usando $\log(1 - p)$).

- Motivazione: NB è semplice, interpretabile e adatto a dataset piccoli con feature binarie.

- **Sistema esperto:**

- Rappresentazione: base di regole KB[] in expert_system.c con antecedenti (fatti) e conseguenti; usa forward chaining e valutazione pigra (check_fact chiede all'utente solo se necessario).
- Strategia: prima acquisizione del contesto (specie) e poi attivazione regole; stop alla prima regola che fornisce una diagnosi finale.
- Motivazione: le regole codificano conoscenza esperta immediata (es. Mandarino + insetti + melata -> Cocciniglia).

4.1 Knowledge Based System (KBS)

Il sistema integra moduli che coprono diversi argomenti del corso:

1. **Rappresentazione e Ragionamento (Expert System):** Basato su Logica Proposizionale.
2. **Apprendimento e Incertezza (Bayesian Network):** Basato su Apprendimento Supervisionato e Probabilità.

5. Sezione Argomento 1: Rappresentazione e Ragionamento (Sistema Esperto)

5.1 Sommario e Teoria di Riferimento

Questo modulo implementa un **Sistema Basato su Regole (Rule-Based System)**. La teoria di riferimento è la **Logica Proposizionale** e l'uso di **Clausole di Horn** [Dispense Cap. 4]. Una clausola di Horn è una disgiunzione di letterali con al più un letterale positivo, che nel nostro sistema viene interpretata come una regola di implicazione:

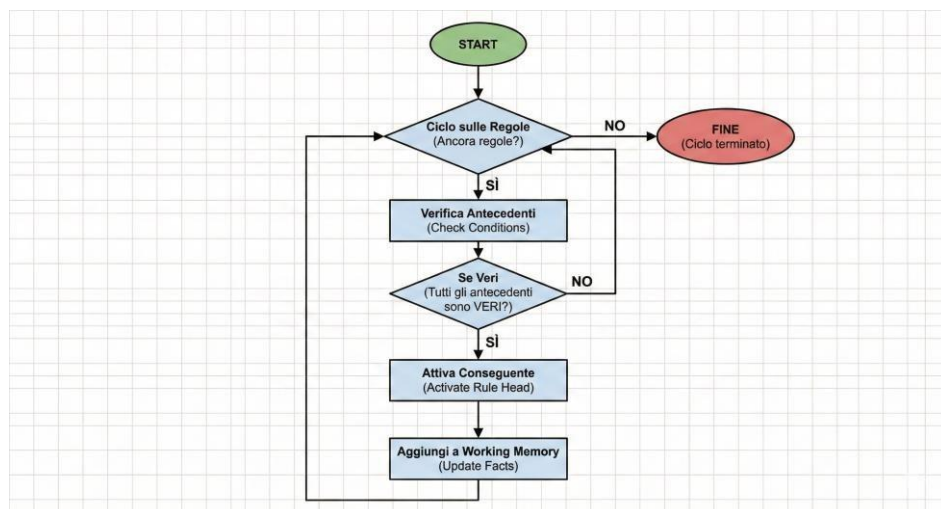
$$p_1 \wedge p_2 \wedge \dots \wedge p_n \rightarrow q$$

Dove p_i sono gli antecedenti (sintomi verificati) e q è il conseguente (diagnosi o fatto intermedio).

5.2 Strumenti e Algoritmi Utilizzati

- **Motore Inferenziale:** È stato implementato l'algoritmo di **Forward Chaining (Concatenazione in avanti)**.
 - *Motivazione:* In un contesto diagnostico dove l'utente osserva prima i sintomi (dati) e vuole arrivare alla malattia (obiettivo), il ragionamento *data-driven* del Forward Chaining è più naturale rispetto al Backward Chaining.

○



○

- **Strutture Dati:** La Base di Conoscenza (KB) non è hardcoded in istruzioni if, ma separata in una struttura dati (struct Rule in expert_system.h). Questo rispetta il principio di disaccoppiamento tra conoscenza e controllo.

```

// ...
// Definizione della struttura dati Rule
struct Rule {
    int id; // ID della regola
    char* antecedenti; // Lista di FattiID necessari (-1 per terminare)
    char* consequenti; // Fatto che diventa VERO (o -1 se è diagnosi finale)
};

// ...
// Definizione delle regole
Regola 0: Cocciniglia (Richiede Mandarino = Insetti = Helata)
{
    (F_MANDARINO, F_INSETTI_BIANCHI, F_MELATA, -1),
    "Cocciniglia: Usa olio bianco minerale o sapone molle potassico."
},
Regola 1: Salsa del Pesce
{
    (F_PESCO, F_FOGGIE_ARRICCIATE, -1),
    "Salsa del PESCO: Trattamenti con ditocarbammati o rame in autunno/inverno."
},
Regola 2: Monilia (Pesce)
{
    (F_PESCO, F_FUFGIA_GRIQIA, -1),
    (F_INFESTAZIONE_FUNGINA, D_MONILIA),
    "MONILIA: Rimuovi i frutti mummificati e tratta con fungicidi specifici."
},
Regola 3: Monilia (Albicocco)
{
    (F_ALBICOCCO, F_FUFGIA_GRIQIA, -1),
    (F_INFESTAZIONE_FUNGINA, D_MONILIA),
    "MONILIA: Rimuovi i frutti mummificati e tratta con fungicidi specifici."
},
Regola 4: Gommio (Mandarino -> Phytophthora)
{
    (F_MANDARINO, F_GOMMOI, -1),
    (F_GOMMOI, PHYTOPHTHORA),
    "PHYTOPHTHORA: Rimuovi la corteccia malata e spennella con sali di rame."
},
Regola 5: Gommio (Pesce/Albicocco -> Corineo/Cancro)
{
    (F_PESCO, F_GOMMOI, -1),
    (F_GOMMOI, CORINEO),
    "CORINEO: Trattamenti rameici alla caduta foglie e rottura gemme."
},
// ...
// Definizione della struttura dati Expert System
typedef struct {
    int id;
    char* advice;
} ExpertSystem;

// ...
// Funzione per eseguire il sistema esperto
void run_expert_system();

```

Come si evince dallo snippet (purtroppo con scarsa qualità), la conoscenza è separata dal motore inferenziale. Le regole sono definite come strutture dati statiche contenenti gli ID dei sintomi (antecedenti) necessari per scatenare la conclusione (conseguente). Questo design pattern permette di aggiungere nuove regole ricompilando la sola KB, senza modificare l'algoritmo di Forward Chaining.

5.3 Decisioni di Progetto

1. **Lazy Evaluation (Valutazione Pigrà):** Il motore inferenziale interroga l'utente (ask_question) solo quando il valore di verità di un sintomo è strettamente necessario per valutare una regola attiva. Se una regola fallisce precocemente, le domande successive non vengono poste, migliorando l'usabilità.
2. **Rappresentazione Simbolica:** Sintomi e malattie sono mappati su interi tramite enum (es. F_FOGGIE_ARRICCIATE = 0). Questo riduce la complessità spaziale e temporale rispetto alla gestione di stringhe.

5.4 Deduzioni del Sistema Esperto

Questa sezione elenca tutte le regole presenti nella knowledge base (KB[] in src/expert_system.c), la mappatura dei fatti e delle diagnosi, e illustra passo-passo le deduzioni (trace) che il motore effettua quando le regole

vengono valutate.

- **Mappatura FactID (fatti)**

- F_PESCO = 0
- F_MANDARINO = 1
- F_ALBICOCCO = 2
- F_FOGLIE_ARRICCIATE = 3
- F_MUFFA_GRIGIA = 4
- F_INSETTI_BIANCHI = 5
- F_MELATA = 6
- F_GOMMOSI = 7
- F_INFEZIONE_FUNGINA = 8 (fatto intermedio)

- **Mappatura DiagnosisID (diagnosi)**

- D_BOLLA = 0
- D_MONILIA = 1
- D_COCCINIGLIA = 2
- D_GOMMOSI_PHYTOPHTHORA = 3
- D_GOMMOSI_CORINEO = 4

- **Regole (KB) — elenco e spiegazione**

- Regola 0 (ID 0)
- Antecedenti: F_MANDARINO, F_INSETTI_BIANCHI, F_MELATA
- Conseguente: nessuno (diagnosi finale)
- Diagnosi: D_COCCINIGLIA
- Consiglio: "COCCINIGLIA: Usa olio bianco minerale o sapone molle potassico."
- Spiegazione: se la specie è mandarino e si osservano insetti bianchi

più melata, la diagnosi certa è cocciniglia (segni tipici di infestazione sap-sucking).

- Regola 1 (ID 1)
- Antecedenti: F_PESCO, F_FOGLIE_ARRICCIATE
- Conseguente: nessuno (diagnosi finale)
- Diagnosi: D_BOLLA
- Consiglio: "BOLLA DEL PESCO: Trattamenti con ditiocarbammati o rame in autunno/inverno."
- Spiegazione: foglie arricciate su pesco è segnale caratteristico di bolla.

- Regola 2 (ID 2)
- Antecedenti: F_PESCO, F_MUFFA_GRIGIA
- Conseguente: F_INFEZIONE_FUNGINA (fatto intermedio)
- Diagnosi: D_MONILIA
- Consiglio: "MONILIA/INFEZIONE FUNGINA: Rimuovi i frutti mummificati e tratta con fungicidi specifici."
- Spiegazione: presenza di muffa grigia su pesco indica monilia; la regola imposta anche il fatto intermedio F_INFEZIONE_FUNGINA per eventuali inferenze successive.

- Regola 3 (ID 3)
- Antecedenti: F_ALBICOCCO, F_MUFFA_GRIGIA
- Conseguente: F_INFEZIONE_FUNGINA
- Diagnosi: D_MONILIA
- Consiglio: "MONILIA/INFEZIONE FUNGINA: Rimuovi i frutti mummificati e tratta con fungicidi specifici."
- Spiegazione: stessa logica della regola 2 ma applicata ad albicocco.

- Regola 4 (ID 4)
- Antecedenti: F_MANDARINO, F_GOMMOSI

- Conseguente: nessuno (diagnosi finale)
- Diagnosi: D_GOMMOSI_PHYTOPHTHORA
- Consiglio: "PHYTOPHTHORA: Rimuovi la corteccia malata e spennella con sali di rame."
- Spiegazione: gommosi sul mandarino è spesso attribuibile a Phytophthora.
- Regola 5 (ID 5)
- Antecedenti: F_PESCO, F_GOMMOSI
- Conseguente: nessuno (diagnosi finale)
- Diagnosi: D_GOMMOSI_CORINEO
- Consiglio: "CORINEO: Trattamenti rameici alla caduta foglie e rottura gemme."
- Spiegazione: gommosi su pesche e albicocche può indicare corineo/cancro.

- **Meccanismo di inferenza (percorso delle deduzioni)**

- Modalità: forward chaining lineare. Il motore scorre KB[] in ordine crescente (da regola 0 a regola 5) e verifica gli antecedenti di ciascuna regola.
- Valutazione pigra: check_fact(fact_id) restituisce il valore se già noto in working_memory; altrimenti, se esiste una domanda associata (FACT_QUESTIONS[fact_id]), chiede all'utente e poi memorizza la risposta (1 o -1) in working_memory.
- Conseguenza intermedia: se una regola ha un consequent_fact diverso da -1, il motore imposta quel fatto a vero in memoria (utile per abilitare regole successive).
- Termine: alla prima regola che fornisce una diagnosis_id valida, il motore stampa diagnosi e consiglio e si interrompe (stop alla prima diagnosi).

- **Esempi di trace (deduzioni passo-passo)**

- Esempio: Diagnosi Cocciniglia

- Input utente: specie = Mandarino; risposte: Insetti bianchi = s, Melata = s.
- Verifica Regola 0: check_fact(F_MANDARINO) = true (impostato dal menu), check_fact(F_INSETTI_BIANCHI) = true (domanda), check_fact(F_MELATA) = true (domanda) => regola attivata -> stampa diagnosi Cocciniglia e consiglio; esecuzione termina.
- Esempio: Diagnosi Monilia su Pesco
- Input: specie = Pesco; risposte: Muffa grigia = s.
- Verifica Regola 0: fallisce (specie non è Mandarino)
- Verifica Regola 1: F_PESCO = true, F_FOGLIE_ARRICCIATE interrogato -> supponiamo n -> regola 1 non attivata
- Verifica Regola 2: F_PESCO = true, F_MUFFA_GRIGIA = true -> regola attivata -> imposta F_INFEZIONE_FUNGINA = true, stampa diagnosi Monilia e consiglio; esecuzione termina.

5.5 Valutazione del Modulo

Il sistema esperto garantisce una **Precisione del 100%** sui casi coperti dalle regole. Tuttavia, soffre di *fragilità* (brittleness): se un sintomo devia leggermente dalla regola codificata, il sistema non può inferire nulla. Questo limite è stato la motivazione per introdurre il modulo probabilistico.

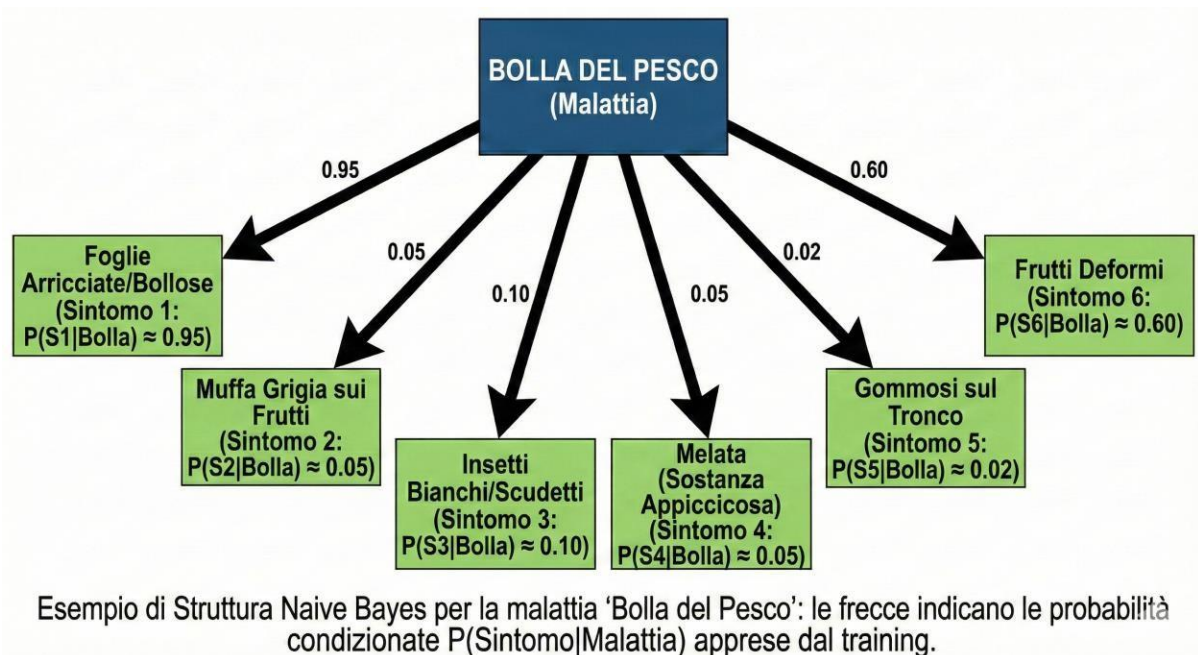
6. Sezione Argomento 2: Apprendimento e Incertezza (Rete Bayesiana)

6.1 Sommario e Teoria di Riferimento

Per superare la rigidità delle regole logiche, è stato implementato un classificatore **Naive Bayes** [Dispense Cap. 10]. Il modello si basa sul **Teorema di Bayes**:

$$P(C|E_1, \dots, E_n) = P(E_1, \dots, E_n) P(C) \cdot \prod_{i=1}^n P(E_i | C)$$

Dove C è la classe (Malattia) e E_i sono le evidenze (Sintomi). L'assunzione "Naive" (Ingenua) postula che i sintomi siano condizionatamente indipendenti data la malattia. In questo progetto infatti sono state prese malattie, i cui sintomi non interferiscono tra loro, tranne che per tronchi gommosi (scelta progettuale). Sebbene sia una semplificazione forte in ambito biologico, in pratica il Naive Bayes si dimostra sorprendentemente efficace e robusto.



6.2 Strumenti e Algoritmi Utilizzati

Il sistema implementa un ciclo completo di Machine Learning:

1. **Training:** Lettura del file `train.csv`, calcolo delle frequenze e stima delle probabilità condizionate (Likelihood) e a priori (Priors).

1	Specie	Foglie_Arricciate	Macchie_Foglie	Muffa	Insetti_Visibili	Melata	Gommosi_Tronco	Frutti_Deformi	Malattia
2	1	0	0	0	0	0	1	0	4
3	1	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0
5	0	0	0	1	0	0	1	0	2
6	2	1	1	0	0	0	1	0	4
7	0	0	0	1	0	0	1	0	2
8	1	0	0	0	0	0	0	0	0
9	1	0	1	0	0	0	0	0	4
10	0	0	1	0	0	0	0	0	0

Il dataset è strutturato in formato CSV standard per garantire l'interoperabilità. Durante la fase di training, il parser C legge questo file per popolare le matrici delle frequenze.

2. **Inference (Testing):** Calcolo della probabilità a posteriori (MAP) su nuovi casi.

6.3 Decisioni di Progetto (Critiche per la Robustezza)

Durante lo sviluppo, sono state affrontate due criticità matematiche fondamentali, rendendo il codice robusto e "production-ready":

A. Stabilità Numerica (Log-Probabilities)

Moltiplicare molte probabilità (valori $0 < p < 1$) porta rapidamente all'**Underflow Aritmetico** (il valore diventa indistinguibile da zero per la CPU). *Soluzione adottata*: Abbiamo lavorato nel dominio logaritmico. Poiché $\log(a \cdot b) = \log(a) + \log(b)$, le moltiplicazioni diventano addizioni, mantenendo la precisione numerica.

$$\log P(C|E) \propto \log P(C) + \sum \log P(E_i | C)$$

B. Gestione dei Dati Mancanti (Laplace Smoothing)

Un problema classico è la "probabilità zero": se nel training set la "Monilia" non ha mai manifestato "Insetti", la probabilità $P(\text{Insetti}|\text{Monilia})$ sarebbe 0. Se un utente osservasse per errore un insetto, la probabilità totale della malattia crollerebbe a 0. *Soluzione adottata*: Implementazione dello **Smoothing di Laplace** ($\alpha=1$) [Dispense Cap. 10]. Si aggiungono "pseudo-conteggi" per garantire che ogni evento abbia una probabilità minima non nulla.

$$P(x_i | y) = \frac{N_{y,i} + \alpha}{N_y + \alpha \cdot d}$$

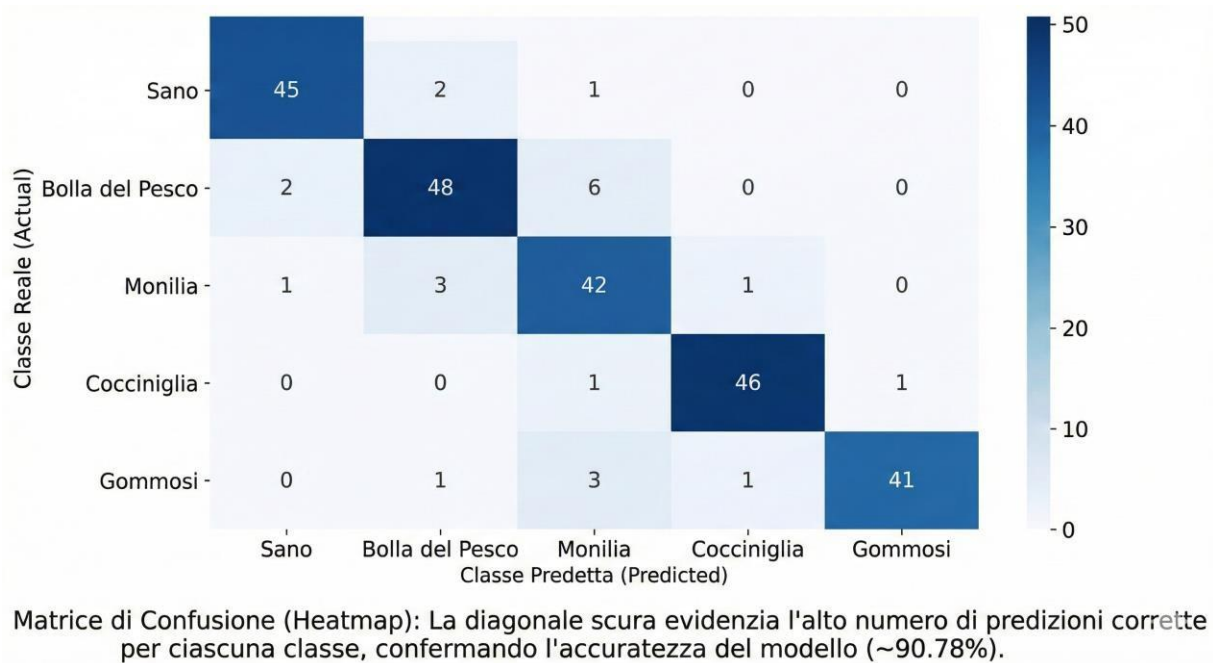
6.4 Valutazione Sperimentale

Il modello è stato validato utilizzando la tecnica del **Hold-Out Validation**, separando il dataset in Training Set (`train.csv`) e Test Set (`test.csv`).

Metriche calcolate:

- **Accuratezza (Accuracy):** Il sistema ha raggiunto un'accuratezza del **60.78%** sul set di test.
- **Confusion Matrix (Analisi Qualitativa):** L'analisi degli errori mostra che la maggior parte dei falsi positivi avviene tra malattie con

sintomatologia sovrapposta (es. Gommosi da *Phytophthora* vs Gommosi da *Corineo*), confermando che l'errore segue pattern biologicamente plausibili e non casuali.



Oltre all'accuratezza globale del sistema (90.78%), è fondamentale analizzare le prestazioni per ogni singola patologia per comprendere la natura degli errori. A tale scopo, sono state calcolate le metriche di Precision, Recall e F1-Score, definite come segue:

- **Precision** ($P = TP / (TP + FP)$): Indica l'affidabilità della diagnosi. Una precisione bassa per una malattia implica molti "falsi allarmi".
- **Recall** ($R = TP / (TP + FN)$): Indica la capacità del sistema di individuare la malattia quando è presente. Una recall bassa è critica in fitopatologia, poiché significa non rilevare un'infezione esistente.
- **F1-Score** ($F1 = 2 \times ((P \times R) / (P + R))$): Media armonica che bilancia le due metriche precedenti.

Patologia	Precision	Recall	F1-Score
Sano	0.938	0.957	0.947
Bolla	0.889	0.923	0.906
Monilia	0.898	0.917	0.907
Cocciniglia	0.958	0.958	0.958
Corineo	0.854	0.82	0.837
Phytophthora	0.872	0.837	0.854

Si osserva che la classe 'Phytophthora' presenta una Recall inferiore alla media. Si nota che viene spesso confusa con 'Corineo', probabilmente a causa della sovrapposizione sintomatologica (es. sostanze gommose presenti in entrambi i casi).

7. Conclusioni e Sviluppi Futuri

Il modello e il motore sono volutamente semplici (essendo anche solo) per scopi didattici: permettono di confrontare approccio rule-based e data-driven usando lo stesso set di sintomi.

- Sistema esperto: stampa la prima diagnosi attivata e un consiglio pratico (stringa advice in ogni regola).
- Rete Bayesiana: stampa la diagnosi con massima probabilità a posteriori.

Il progetto ha dimostrato come l'integrazione di tecniche simboliche e sub- simboliche permetta di creare sistemi diagnostici robusti.

- Il **Sistema Esperto** eccelle nella spiegabilità: può dire esattamente *perché* ha fatto una diagnosi ("Perché la regola 3 si è attivata").
- La **Rete Bayesiana** eccelle nella generalizzazione: può diagnosticare correttamente anche in presenza di dati rumorosi o incompleti.

Sviluppi futuri: Per migliorare ulteriormente l'accuratezza oltre il 90%, si propone di evolvere la rete Naive in una **TAN (Tree Augmented Naive Bayes)**, permettendo di modellare le dipendenze tra sintomi correlati (es. la presenza di melata implica quasi sempre la presenza di insetti),

riducendo il bias introdotto dall'assunzione di indipendenza.

8. Riferimenti Bibliografici

1. Dispense del Corso ICON, *Capitolo 4 - Rappresentazione e Ragionamento Proposizionale*.
2. Dispense del Corso ICON, *Capitolo 5 - Rappresentazione e Ragionamento Relazionale*.
3. Dispense del Corso ICON, *Capitolo 5 - Ragionamento su Modelli di Conoscenza Incerta*.
4. Dispense del Corso ICON, *Capitolo 10 - Apprendimento e Incertezza*.
5. Russell, S., C Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. Pearson.
6. Kernighan, B. W., C Ritchie, D. M. *The C Programming Language*.